

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computación

Programa 3:
Estacionamiento de Vehículos

Adrián Mora Rivera
Carné: 2024800149

Grupo 4
I semestre, 2024

Curso de Taller de Programación
Profesor: William Mata Rodríguez

Lunes 17 de junio del 2024

Tabla de contenido

Tema	Página
1. Enunciado del proyecto	1
1.1. Objetivos.....	1
1.2. Definición.....	2
1.3. Requerimientos de información	3
2. Temas investigados	8
2.1. La interfaz gráfica de usuario (GUI) en el desarrollo de software.....	8
2.2. Programación dirigida por eventos	9
2.3. Desarrollo de GUI en Python usando la biblioteca tkinter	10
2.4. Aspectos adicionales para la solución.....	12
3. Conclusiones del trabajo.....	15
4. Estadística de tiempos	18
5. Lista de revisión del proyecto y análisis de resultados	19
6. Manual de usuario	20
7. Bibliografía	21

1. Enunciado del proyecto

El presente documento sirve como la documentación externa para el proyecto Estacionamiento de Vehículos, el cual es el tercer proyecto de desarrollo de programas del curso de Taller de Programación. A continuación, se detalla su enunciado con los objetivos, la definición y los requerimientos del proyecto.

1.1. Objetivos del proyecto

- ❖ Continuar aplicando el ciclo completo de la metodología general de desarrollo de programas a situaciones de mayor alcance:
 - Entender el problema.
 - Diseñar algoritmo.
 - Codificar algoritmo
 - Probar y evaluar programa
- ❖ Aplicar y reforzar aspectos del lenguaje Python 3.
 - Uso de diversos componentes del lenguaje.
 - Desarrollo y reutilización de funciones.
 - Uso de estructuras condicionales y de repetición de procesos.
 - Manejo de la técnica de iteración para repetición de procesos.
 - Uso de estructuras de datos.
- ❖ Uso de archivos.
- ❖ Aplicar buenas prácticas de programación básicas:
 - Documentación interna y externa del programa
 - Nombres significativos
 - Evaluar alternativas
 - Uso de técnica divide y vencerás (dividir el problema en partes, desarrollar cada una de esas partes)
 - Uso de convenciones para la escritura de programas Python (estilo de programación)
- ❖ Validación de los datos de entrada: todos los datos de entrada se deben validar según restricciones que se indican en cada uno de ellos.
- ❖ Fomentar la habilidad de investigación: en el desarrollo de proyectos es común que aparezcan algunos aspectos afines sobre los cuales conocemos poco o nada, para resolver nos queda investigar. Esta situación ha sido expuesta y comprobada en los proyectos anteriores.

Para este proyecto específico los aspectos a investigar son:

- La interfaz gráfica de usuario (GUI: Graphical User Interface) en el desarrollo de software.
- Programación dirigida por eventos.
- Desarrollo de GUI en Python usando la biblioteca tkinter. Esta biblioteca viene incluida en Python y se le considera el estándar de facto para la programación de GUI con Python. Se explican los elementos específicos de tkinter usados para el desarrollo del proyecto.
- Cualquier otro aspecto necesario para ofrecer su solución.
- Cada punto investigado debe tener sus referencias bibliográficas (formato APA).

1.2 Definición del proyecto

Este programa es para el control de operaciones que se hacen en un estacionamiento de vehículos (parqueo de vehículos): las entradas y salidas de los vehículos al estacionamiento, un cajero automático para hacer los cobros por el servicio, un reporte de ingresos de dinero y el registro de un historial del uso del estacionamiento.

Cada vehículo tiene una placa como identificación única (string de 1 a 8 caracteres).

El programa tendrá una **interfaz gráfica de usuario (GUI: Graphical User Interface)** que inicia con un menú principal desde el cual se accederá su funcionalidad, es decir, las diferentes operaciones que el programa hace. Se puede agregar otras funcionalidades que vayan a mejorar el producto. Se puede hacer cambios en la interfaz gráfica previamente acordados con el profesor.

El programa usará archivos para almacenar la información.

Para manejar cada espacio del estacionamiento se usará la lista **parqueo**: cada elemento será una sublista que corresponde a un espacio del estacionamiento. Inicialmente todos los elementos van a estar vacíos. Por ejemplo, si la cantidad de espacios es 10 la lista tendrá 10 sublistas vacías: [[], [], [], [], [], [], [], [], [], []]. El primer elemento de la lista es el espacio número 1 del estacionamiento, el segundo elemento es el espacio número 2 y así sucesivamente hasta el espacio n.

Cuando un vehículo necesita ingresar se busca en la lista **parqueo** el primer espacio disponible para asignarlo y registrar los datos de placa y tiempo de entrada.

Estructura de la lista **parqueo**:

```
[ [ placa, fecha_hora_entrada, fecha_hora_pago, valor_pagado ], ... ]
```

Elementos de `fecha_hora_entrada` y `fecha_hora_pago`: día, mes, año, hora, minutos, cada uno de estos datos deben ser válidos, por ejemplo, no se permite un 35 de enero. Para manejar este tipo de datos debe usar algún módulo que facilite su manipulación.

Con la entrada de un vehículo `fecha_hora_pago` estará vacío y `valor_pagado` estará en cero.

Cuando un vehículo paga se actualiza su respectivo elemento en la lista **parqueo** con los datos reales de `fecha_hora_pago` y `valor_pagado`.

Cuando un vehículo sale físicamente se actualiza la lista **parqueo**: el elemento se cambia por una lista vacía para indicar que el espacio está libre, el elemento eliminado se agrega a la lista **historial_parqueo** con la siguiente estructura:

```
[ [placa, numero_espacio, fecha_hora_entrada, fecha_hora_pago,
valor_pagado, fecha_hora_salida], ... ]
```

Note que la lista **historial_parqueo** contiene un elemento para cada pago hecho por los vehículos que salieron físicamente.

1.3 Requerimientos de información

- ❖ Lo primero que desplegará la interfaz gráfica será una barra de menú (similar al menú del IDLE de Python) con estas funcionalidades:

- Configuración
 - Dinero del cajero (submenú)
 - Saldo del cajero
 - Cargar cajero
- Entrada de vehículo
- Cajero
- Salida de vehículo
- Reporte de ingresos de dinero
- Ayuda
- Acerca de
- Salir

- En este menú se puede salir del programa por medio de la opción Salir de la barra de menú o con la opción estándar Cerrar en la GUI ("X").
- ❖ Seguidamente se explica el detalle de las funcionalidades.

1. **CONFIGURACIÓN**, esta funcionalidad permite establecer una serie de parámetros necesarios para la operación del programa. Sin estos datos el resto de la funcionalidad del programa no puede usarse. Dichos parámetros son:
 - Cantidad de espacios en el parqueo
 - Precio por hora
 - Pago mínimo
 - Redondear el tiempo cobrado al próximo minuto
 - Minutos máximos para salir después del pago
 - Tipos de moneda (máximo 3)
 - Tipos de billetes (máximo 5).

2. **DINERO DEL CAJERO**, Esta funcionalidad se usa para administrar el dinero que tiene el cajero automático. Es un submenú con dos opciones:
 - 2.1. **Opción Saldo del cajero**: permite visualizar los saldos por denominación que tiene el cajero. Opcionalmente puede vaciar el cajero, es decir, dejar todas sus denominaciones en cero.
 La sección ENTRADAS contiene las cantidades que han entrado al cajero por los conceptos de cargas y pagos.
 La sección SALIDAS contiene las cantidades que han salido por concepto de cambios (vuelto).
 La sección SALDO contiene ENTRADAS – SALIDAS.
 Opción Vaciar cajero: cuando se ingresa a esta ventana esta opción permanece inactiva. El usuario puede activar la opción (con un check) lo cual significa que todos los valores se pondrán en cero cuando se seleccione el botón Ok

 - 2.1. **Opción Cargar cajero**: permite actualizar los montos de dinero de las diferentes denominaciones para dar cambios (vuelto) a los clientes. Las columnas de la sección SALDO se actualizan conforme se van registrando datos en la columna CANTIDAD de la sección CARGA. Inicialmente el cajero empieza con cero en todos los saldos de cantidades de las diferentes denominaciones. Los saldos se van modificando con las cargas, los pagos y los cambios. Las cargas se pueden hacer en cualquier momento y cuantas veces se necesiten.

3. **ENTRADA DE VEHÍCULO**, opción que se usa para que un vehículo pueda entrar al estacionamiento. Puede entrar al estacionamiento si hay espacio disponible (elementos con una lista vacía) en cuyo caso se registran los datos en la lista parqueo. Aquí es donde el usuario ingresa el dato placa

4. **CAJERO**, los clientes pagan por el servicio de estacionamiento usando esta parte que es una simulación de un cajero automático. Se pide la placa e inmediatamente se procede a desplegar los datos de la ventana y el cálculo del monto a pagar.

El pago se puede hacer con: monedas, billetes y tarjeta de crédito. Para simular el pago en efectivo hay botones de MONEDAS y BILLETES. Cada vez que se presiona un botón de dinero inmediatamente se acumula y actualiza en la ventana el monto correspondiente en la casilla Pago.

En caso de pagar con una tarjeta de crédito debe posicionar el cursor en esa casilla y dar un número natural de 10 dígitos exactos, tomando de la tarjeta la diferencia que exista entre A PAGAR y Pago. No hay validación de aceptación de la tarjeta de crédito, vamos a asumir que siempre se hace.

5. **SALIDA DE VEHÍCULO**, que servirá para que el espacio ocupado por el vehículo se libere hasta que dicho vehículo haga la salida física. Esto es debido a que puede hacer el pago, pero permanecer físicamente en el estacionamiento. Por eso mientras el vehículo no haga este proceso, se asume que sigue permaneciendo en el estacionamiento, ocupando el mismo espacio.

Cuando no pueda hacer la salida porque excedió el tiempo para ello, automáticamente la lista parqueo se actualiza en el mismo elemento con los datos de fecha y hora de entrada tomados de la fecha y hora de pago, reflejando que es una nueva entrada de vehículo para que pueda hacer el pago en el momento que considere oportuno.

6. **REPORTE DE INGRESOS DE DINERO**, que permite calcular los ingresos reales de dinero que ha obtenido el estacionamiento. Para hacer el cálculo el usuario digita un periodo de tiempo. Los pagos de los vehículos que no han salido físicamente están en la lista parqueo. Los pagos de los vehículos que han salido físicamente están en la lista **historial_parqueo**. Opcionalmente permite calcular un estimado de ingresos de dinero considerando los vehículos que están en el estacionamiento pero que aún no han pagado: estos se encuentran en la lista parqueo y no tienen datos de pago. Los cálculos se hacen tomando el tiempo transcurrido desde su entrada hasta fecha y hora que el usuario registre para la estimación. Si el usuario no ocupa esta estimación la fecha y hora los deja en blanco.

7. **AYUDA**, opción usada para que el usuario pueda ver el Manual de Usuario directamente en la computadora (despliega el archivo de la documentación).
8. **ACERCA DE**
 - ❖ Esta opción desplegará los datos de:
 - nombre del programa (Estacionamiento de Vehículos)
 - la versión (Versión 1.0.0)
 - la fecha de creación (lunes 17 de junio del 2024)
 - el autor de este proyecto (Adrián Mora Rivera)
9. **SALIR**, esta opción sirve para salir del programa. También se puede salir con la opción Cerrar ("X") en la GUI.

❖ Uso de archivos y de estructuras de datos

Como requerimiento final, para este programa la información manejada es almacenada en archivos. El uso de archivos usará el esquema del programa anterior.

Cada vez que inicie la ejecución del programa debe leer los datos que están en los archivos para asignarlos a las estructuras en memoria. La primera vez que se corre el programa los archivos están vacíos, luego estos se van actualizando de acuerdo a las operaciones que haga el usuario.

Cada vez que termina la ejecución del programa debe grabar los datos en los archivos.

- ✓ **ARCHIVO configuración.dat**
 - Contiene los datos de configuración.
 - Debe ser un archivo tipo string por líneas: cada dato de configuración se debe poner en una línea.
- ✓ **ARCHIVO parqueo.dat**
 - Contiene la lista parqueo. Es un archivo de tipo binario.
- ✓ **ARCHIVO historial_parqueo.dat**
 - Contiene la lista historial_parqueo. Es un archivo de tipo binario.
- ✓ **ARCHIVO cajero.dat**
 - Contiene los datos para el manejo del cajero. Es un archivo de tipo binario.

Adicionalmente, se debe usar las estructuras de datos indicadas en esta especificación, tanto en nombre como en funcionalidad.

Adicionalmente a los archivos y estructuras de datos indicadas en este proyecto, se pueden utilizar otros según sus algoritmos.

2 Temas investigados

A continuación, se detallan todos los temas investigados para llevar a cabo el presente proyecto.

2.1 La interfaz gráfica de usuario (GUI) en el desarrollo de software

En los inicios de la programación, el software era complejo de entender, provocando que solo aquellos con conocimientos en el área los pudieran emplear. Años después, luego de avances importantes, los programas contemporáneos son visualmente atractivos, fáciles de utilizar y muy intuitivos. Esto se debe a que se manejan en interfaces de usuario diseñadas para simplificar su uso. Tales interfaces se llaman GUI (o interfaz gráfica de usuario).

Según la literatura, llegar a estas interfaces tomó años; tomando inicio en la década de los setenta con la empresa Xerox, que logró el manejo de computadoras “con el ratón y el teclado en lugar de solo con instrucciones en formato de texto. Xerox Alto fue el primer PC con una interfaz gráfica. Apple le siguió en los años 80 con el Macintosh” (Ionos, 2020, párr. 1). A partir de ahí, ha sido exponencial el progreso, llegando a hitos como los *smartphones*, tablets, relojes inteligentes y demás dispositivos cuyas interfaces los permiten utilizar con simples gestos y movimientos de dedos

Ahora bien, por definición, la GUI es una interfaz gráfica que busca “representar el código del backend de un sistema de la forma más clara posible para el usuario [haciendo uso de] los iconos y las imágenes, ya que solo estos permiten una aplicación universal e independiente del texto” (Ionos, 2020, párr. 3). Además, se usan menús y otros símbolos visuales que representan objetos, funciones o características del sistema. Hoy en día, casi todos los programas vienen con esta interfaz.

Por otro lado, en su artículo, la Escuela Superior de Diseño de Barcelona (2023) explica que la GUI parte de la interacción entre el *mouse* y el teclado: al mover el ratón, se desplaza el puntero y la señal del dispositivo se transmite al ordenador, por lo que ese movimiento equivale a una instrucción por ejecutar. Por ejemplo, si se lleva el puntero a un icono en el menú del programa, se ejecutan sus funciones asociadas. Esto es de gran utilidad ya que, sin la GUI, se tendrían que utilizar líneas de comandos para llamar a programas y aplicaciones.

Asimismo, la literatura menciona que la interfaz gráfica se compone de los siguientes elementos:

- Campos de entrada (para que el usuario digite datos)
- Ventana (es el contenedor principal de la interfaz)
- Lienzo o canvas (que dibuja gráficos)

- Marcos (agrupan y organizan los widgets)
- Botones (ejecutan acciones al darles clic encima)
- Bloques de encabezado (para mostrar títulos o etiquetas)
- Campos de texto (áreas para ingresar o mostrar textos largos)

También, se destaca cómo, además de esos elementos, la GUI se basa en representar de forma ilustrativa todo lo que se necesite comunicar al usuario. Al respecto, la literatura expone el ejemplo de la papelera de reciclaje: “su representación gráfica tanto en Windows como en Mac es la imagen de una papelera real. De este modo, el usuario sabe de inmediato que sirve para borrar documentos y archivos” (Ionos, 2020, párr. 7).

Para finalizar, el desarrollo de las GUI tiene que ser un proceso en el que se dé atención al detalle para que se logre el objetivo de crear una interfaz intuitiva. Parafraseando la literatura, esta clase de programas deben desplegar solo la necesaria, para optimizar al máximo la experiencia de usuario.

Además, al pasar de una pantalla a otra, es necesario que se mantenga el mismo tono, estilo, diseño y paleta de colores para brindarle coherencia al programa como un todo. Por último, el diseño tiene que ser ergonómico, familiar y con elementos que el usuario conoce, como botones, barras de menús e iconos (Ionos, 2020).

2.2 Programación dirigida por eventos

Para que la interfaz gráfica alcance brindar simplicidad al usuario al entender la navegación del sistema y de cómo ejecutar sus funciones, se tiene que adaptar la estructura de los programas ya que, según un artículo por IONOS Digital Guide (2020), “las GUI no se pueden programar linealmente, sino que deben escribirse de tal manera que un comando solo se ejecute cuando el usuario dé la señal para este” (párr. 8). Por este motivo, se usa la programación basada en eventos.

Un artículo por la escuela Programación Desde Cero, señala que este tipo de programación es un paradigma en el que el flujo del programa se basa en eventos, el cual se define como “una acción que se desencadena como resultado de una interacción entre un objeto y el usuario o el sistema. En términos simples, un evento es una respuesta a una acción” (s.f.). Dicho de otra manera, cada vez que en el programa se ejecute alguna instrucción porque el usuario hizo, por ejemplo, clic en un botón, un evento es generado.

Añadiendo a lo anterior, los eventos son útiles en el desarrollo de programas GUI porque, con la existencia de botones y otros componentes, se enlazan las acciones que el usuario hace con esos elementos a funciones específicas. Además, son de alta conveniencia para la comunicación usuario-sistema. La literatura lo ilustra con este caso: “un evento puede ser desencadenado cuando se ha completado una operación en segundo plano, lo que hace que la aplicación notifique

al usuario de que la operación ha sido completada con éxito” (Programación Desde Cero, s.f., párr. 5).

Por otra parte, este tipo de programación implementa un bucle principal o *main loop*. Según la literatura, estos bucles serán incluidos en la lógica de los disparadores – es decir, un evento específico que inicia la ejecución de una función el sistema, tal como un clic o la pulsación de una tecla –, que tienen el rol de

analizar los eventos para determinar su naturaleza y entonces enviarlos al manejador adecuado que es capaz de trabajar con eventos de esa naturaleza. El disparador tiene que procesar un flujo de eventos, así que su lógica debe incluir un bucle de eventos para poder enviar un evento a un manejador y volver a escuchar a la espera de nuevos eventos que disparar.

Es común que exista un evento especial que rompa el bucle y salga de la aplicación, a ese evento se le llama evento finalizador y es muy común en todas las librerías para escribir aplicaciones GUI. También puede ocurrir que el disparador capture un evento de naturaleza desconocida o para el que no exista un manejador adecuado, en esos casos, el disparador debe descartar el evento o lanzar una excepción (Campos, 2011, párr. 8).

Como resultado, las GUI basadas en eventos responden inmediatamente a las acciones del usuario, brindando una experiencia interactiva y dinámica. Usar eventos, además, permite un manejo eficiente de operaciones asíncronas como la entrada y salida de red o archivos para, en vez de bloquear la ejecución esperando una operación, usar controladores de eventos que gestionen la ocurrencia, mejorando así el rendimiento del sistema (Campos, 2011).

En síntesis, la programación basada en eventos es un paradigma que maneja el flujo de ejecución por eventos, que son las acciones que el usuario hace o los mensajes de otros programas, y que aprovecha, de entre otros elementos, manejadores de eventos que responden a tales eventos con funciones específicas.

2.3 Desarrollo de GUI en Python usando la biblioteca Tkinter

Para el desarrollo de interfaces gráficas de usuario en Python, su biblioteca Tkinter es el estándar. Según Martín (2020), Tkinter se trata de “una capa orientada a objetos basada en Tcl (sencillo y versátil lenguaje de programación open-source) y Tk (la herramienta GUI estándar para Tcl)” (párr. 1).

A su vez, esta ofrece una sintaxis simple y directa que ayuda al desarrollo de programas incluso con conocimientos elementales de programación. Cabe destacar que, siendo de Python, dicha biblioteca se integra bien con otras, garantizando

amplias funcionalidades que ayudan a crear aplicaciones portables que carezcan la necesidad de dependencias externas.

Para este proyecto, se importó tk, el cual es un paquete proveniente de Tcl implementado en C que agrega comandos personalizados que permiten manipular widget en la GUI. Cada objeto de la clase Tk agrega su instancia de interprete Tcl con Tk cargado en el. Los widget de Tk son muy personalizables, aunque a costa de una apariencia anticuada. Tk usa eventos de la cola de Tcl para generar y procesar eventos de la GUI. (Python Software Foundation, s.f., párr. 7).

A continuación, se detallan los elementos específicos de Tkinter usados para el desarrollo del proyecto.

- 🌈 Widgets. A primera instancia, se usaron los widgets. La documentación oficial de Python explica que estos son componentes gráficos fundamentales en la construcción de GUI, representados como objetos Python instanciados desde clases como `ttk.Frame`, `ttk.Label` y `ttk.Button`.

De igual modo, estos se organizan en jerarquías donde cada widget (*child*) se puede contener dentro de otro (*parent*), como una etiqueta y un botón pueden estar adentro de un marco y este puede estar al mismo tiempo dentro del marco raíz de la interfaz. (Python Software Foundation, s.f.).

Por añadidura, los widgets son configurables en cuanto a su apariencia y funcionalidad y deben ser añadidos con administradores de geometrías como el método *grid* (empleado en el proyecto, junto con *pack*).

- 🌈 Tk. Martín (2020) explica que Tk se puede usar como la raíz de la interfaz, por lo que se aplicó para colocar el resto de los widgets del programa.
- 🌈 Frame. A fin de agrupar los diferentes widgets, se usó frame, el cual genera marcos que facilitan su disposición.
- 🌈 Label. Se emplearon para mostrar textos, títulos, encabezados, entre otros. Se tratan de etiquetas que presentan información estática y se les puede configurar el tamaño de fuente, color, alineación y estilo (Python Software Foundation, s.f.).
- 🌈 Entry. Para los datos de configuración, placa, fechas para el reporte de ingresos, entre otros, se usaron este tipo de etiquetas. Son útiles para introducir texto corto como en formularios y, además de mostrar el texto de forma dinámica, se pueden configurar para limitar la cantidad de caracteres introducidos o limitar las teclas válidas asociando variables `StringVar` para

vincular y actualizar automáticamente el contenido del campo (Python Software Foundation, s.f.).

- ✚ Button. Este elemento de Tkinter ejecuta una función asociada al ser pulsado. Fueron empleados a través de todo el programa.
- ✚ Checkbutton. Similar al botón, pero permite elegir de entre opciones propuestas. Se empleó una vez, para la opción de vaciar el cajero.
- ✚ MessageBox. En Tkinter, este widget es conveniente para mostrar mensajes modales predefinidos, como diálogos de alerta, confirmación, error, etc., mediante ventanas emergentes estándar (Python Software Foundation, s.f.). Se utilizaron de tipo error para los casos en que se pusieran denominaciones mayores a la anterior (función de configuración), se intentara acceder a las demás opciones sin configurar los datos iniciales, se quisiera modificar los datos de configuración con un parqueo no vacío, se digitaran fechas u horas incorrectas (función reporte de ingresos de dinero), entre otros.
- ✚ Scrolledtext. Llegada la función 4 de cajero, cuando se notó que el tamaño de todos los *labels* y cajas de entry era muy grande como para acomodar en su totalidad los componentes de esta ventana, se implementó el widget *scrolledtext*. Según la documentación, este brinda barras de desplazamiento integradas que ayudan a que los usuarios alcancen navegar el contenido extenso de una ventana. (Python Software Foundation, s.f.).
- ✚ Menu. Finalmente, para acceder a las funcionalidades del programa y seguir las especificaciones, se empleó el widget Menu de Tkinter para crear la barra de menús, en la que todos los elementos se organizan jerárquicamente con menús principales y submenús (los únicos submenús aplicados fueron para mostrar el saldo del cajero y para cargarlo, el resto de opciones fueron parte del menú principal).

Tkinter es una herramienta muy útil y bastante completa, idónea para el desarrollo de programas GUI en Python. Con estos elementos, se logró aprovechar al máximo lo que tiene para ofrecer y así crear un producto de calidad.

2.4 Aspectos adicionales para la solución

Este proyecto incluye las funcionalidades de Entrada de vehículo, Cajero, Salida de vehículo y Reporte de ingresos de dinero, las cuales hacen uso de datos de tiempo. En específico, cuando se va a registrar un vehículo del estacionamiento, así como cuando se va a pagar y a retirar, es necesario llevar un control de la fecha,

hora, día y minutos en las que se realizaron estas acciones. Por eso, fue necesario para la solución investigar sobre algún módulo que facilitara este proceso.

Consecuentemente, se consultó la documentación oficial de Python y se dio con el módulo `datetime`, el cual “proporciona clases para manipular fechas y horas. Si bien la implementación permite operaciones aritméticas con fechas y horas, su principal objetivo es poder extraer campos de forma eficiente para su posterior manipulación o formateo” (Python Software Foundation, s.f., párr. 1).

Luego de importar `datetime`, se utilizó `timedelta` y el paquete `dateutil` (una biblioteca de terceros con zona horaria ampliada y soporte de análisis).

En primer lugar, la clase `datetime.timedelta` fue empleada para las funciones de cajero y salida de vehículo, con el fin de averiguar, junto con `relativedelta`, el tiempo transcurrido desde la hora de entrada hasta la hora del pago y desde la hora de pago a la hora de salida, respectivamente. Según la literatura, `timedelta` está diseñada para calcular una “duración que expresa la diferencia entre dos instancias `date`, `time` o `datetime` a una resolución de microsegundos” (Python Software Foundation, s.f., párr. 14).

Además, los argumentos, que pueden ser enteros o flotantes negativos o positivos, se convierten a las siguientes unidades:

- Un milisegundo a 1000 microsegundos.
- Un minuto a 60 segundos.
- Una hora a 3600 segundos.
- Una semana a 7 días.

De esta manera, los días, segundos y microsegundos se normalizan para que la representación sea única, con

- $0 \leq \text{microsegundos} < 1000000$
- $0 \leq \text{segundos} < 3600 \cdot 24$ (el número de segundos en un día)
- $-999999999 \leq \text{days} \leq 999999999$ (Python Software Foundation).

Aunado a lo anterior, el paquete `dateutil` se empleó para aprovechar sus herramientas de análisis, manipulación y cálculo de fechas y tiempos. Específicamente, cuando se trató de calcular el monto cobrado y los tiempos transcurridos desde el momento de entrada al momento de pago y luego del momento de pago al de la salida, se necesitó esta biblioteca que gestiona zonas horarias y permite hacer cálculos completos como diferencias entre dos fechas (Python Software Foundation).

Como tercer aspecto investigado necesario para la solución, está la biblioteca `Python Imaging Library (PIL)`. Esta es una biblioteca que facilita añadir capacidades de procesamiento de imágenes. Según la documentación, debido a la amplia

compatibilidad de formatos de archivo manejados por la biblioteca, se puede acceder a imágenes y, con otras importaciones, cargar imágenes en widgets de Tkinter como *Label* o *Canvas* (Pillow, s.f.).

En efecto, al importar Image, se logró abrir, manipular y guardar la imagen del fondo e ícono del programa. Encima, con ImageTk se convirtió las imágenes de PIL a un formato compatible con Tkinter para ser mostrados en la interfaz gráfica.

De esta manera, luego de una extensiva revisión de la literatura y documentación disponible, se armó un marco de herramientas necesarias para alcanzar la funcionalidad de cada sección de este tercer programa.

3 Conclusiones del trabajo

Llegado el fin de este proyecto, es posible concluir que, comparado a cómo estaba apenas se nos explicaron los requerimientos de información, el espectro de mis habilidades y conocimientos es ahora mucho más amplio. El programa Estacionamiento de Vehículos me ha dado la oportunidad de desarrollar el amaestramiento en cuanto a tres aspectos: cómo funcionan las interfaces gráficas usando Tkinter, la validación en los widgets Entry y un refuerzo en cuanto al manejo de diccionarios anidados.

Por ende, a continuación, se explica en detalle los problemas presentados durante el proyecto, sus soluciones y todos los aprendizajes obtenidos.

1. Cómo funcionan las interfaces gráficas usando Tkinter

Al inicio, fue complicado para mí entender el concepto de un programa en interfaz gráfica (GUI), ya que me había apegado mucho al esquema de las interfaces TUI. Cuando noté que la función principal ya no seguía el mismo formato que el *while* con *matchcase* empleado en los dos proyectos anteriores, me di cuenta de lo diferente que es la interfaz gráfica.

En efecto, esta diferencia entre TUI y GUI fue un problema debido a la gran cantidad de aspectos que se tienen que ejecutar de una forma distinta a la que ya venía acostumbrado. Por ejemplo, desplegar datos en pantalla. En lugar de ser un simple *print*, me tomó un tiempo adaptarme al uso de etiquetas o *labels*. Luego de familiarizarme con el tema, ahora sé que al crear un widget y configurarle el tamaño y la posición, se puede desplegar un texto en la ventana del programa. Adicionalmente, ahora sé lo convenientes que son por su naturaleza dinámica, es decir, no son estáticos como un *print*. Por el contrario, se puede cambiar lo que dicen, se puede poner que el texto que desplieguen sea el resultado de un cálculo, entre otros. Esto me resultó muy útil para las funciones 2.1, 2.2 y 4 (la del cajero) en las que se debe ir actualizando etiquetas en tiempo real a medida que el usuario va presionando botones.

Sobre botones, algo que se me hizo difícil con Tkinter al inicio fue asignar funciones a los botones, específicamente en cuanto a funciones que tienen parámetros. Debido a que cuando se carece de parámetros basta con escribir *command*, igual, y luego el nombre de la función, estaba aplicando lo mismo para casos en los que sí había parámetros. Por ejemplo, para asignar funciones a la subopción del menú “Dinero del cajero”, se llama a *saldo_cajero(dicc_cajero)*, por lo que lo escribía como *command=saldo_cajero(dicc_cajero)*. Empero, una vez que comprendí que cuando se llama a funciones con parámetro se usa también *lambda*, las comencé a llamar correctamente, haciendo que los errores que me daba desaparecieran y con eso logré dominar el tema.

En tercer lugar, otro aspecto que me dio problemas con respecto a las interfaces gráficas en Tkinter fue el manejo de las filas (*row*) y columnas (*column*). Debido a que usé `grid()` para dividir las ventanas en filas y columnas, hubo casos como en las funciones de mostrar el saldo del cajero y cargar el cajero, en las que no me aparecían botones y *labels* que recién había escrito. Me preguntaba, ¿por qué no se ve este botón? Especialmente, me sucedió con los de las monedas. Posicionaba las filas para mostrar y cargar monedas, luego hacía lo mismo para los billetes, pero al correrlo, solo aparecían los billetes en la posición de las monedas. Me di cuenta de que no estaba actualizando la fila luego de poner los elementos de las monedas, por lo que los billetes se ponían encima de las monedas, en lugar de debajo de estas. Una vez que entendí el problema, aplicar una simple variable para ir controlando en cuáles filas se pone cuál elemento fue suficiente para solucionarlo.

2. Validación en los widgets de Entry

Este programa pudo ser diferente a los anteriores en muchas formas, pero algo que permaneció constante fue la validación de datos. En las funciones para las opciones 1, 2.2, 3, 4, 5 y 6, se tenía que solicitar el usuario ingresar datos. Dependiendo de lo fuera, eran restricciones como que el valor ingresado fuera un entero mayor o igual a 0, que fuera un entero dentro de 0 a 60, que se tratara de un string no mayor a ocho caracteres, entre otros. Por eso, siguiendo el paradigma al que me había acostumbrado con las TUI, pensé que tendría que verificar los datos una vez que el usuario finalizara de escribir todos y presionaba “Ok”.

No obstante, logré aprender que en Tkinter uno puede implementar un estilo de validación de entradas en tiempo real para que, antes de siquiera procesarlos, confirmar que los valores ingresados sean válidos. Para esto, ahora estoy familiarizado con la opción *validate* y *validatecommand* de los widgets para *entry*. Su función es muy práctica: sirven para bloquear las teclas que corresponden a caracteres inválidos según el caso. Por ejemplo, si el valor a ingresar es un entero positivo, se bloquean todas las teclas salvo las numerales. O si el valor puede contener tanto letras como números, pero hasta un máximo de ocho caracteres, entonces apenas se escribe el octavo carácter se bloquea todo el teclado para evitar la digitación en exceso.

De esta manera, ahora conozco esta opción para que, en lugar de esperar a que el usuario acabe de escribir los datos y luego procesarlos – lo cual haría más complejas y extensivas a las funciones del programa –, simplemente se puede controlar lo que escribe para asegurarse de que las variables del sistema nunca manejen valores incorrectos.

3. Manejo de diccionarios anidados

Por último, un problema que se me presentó fue idear la manera en que trabajaría el cajero. Al leer el instructivo y observar que necesitaría almacenar los datos de tal forma que pudiera acceder a cada denominación, la cantidad de entradas, de salidas y el total final (la resta de entradas - salidas) y además saber distinguir entre cuáles eran monedas y cuáles billetes, la opción más rentable a mi parecer fue un diccionario al cual llamé `dicc_cajero`. Este diccionario, a su vez, se compone de dos diccionarios, `dicc_monedas` y `dicc_billetes`, uno para cada grupo de denominaciones.

Iniciando, no hubo complicación con esta estructura; fácilmente podía ir visualizando los datos para cada tipo de moneda o billete. Ahora bien, el problema surgió cuando llegué a la opción 2.2. Cargar cajero. En este punto, me di cuenta de lo complejo que había hecho la estructura: para actualizar la cantidad de monedas de 50, por ejemplo, tenía que ir a `dicc_cajero`, revisar si se trataba de una moneda o un billete, acceder a `dicc_moneda`, buscar la llave correspondiente, como el valor para denominación es una lista con tres sublistas (una para las entradas, otra para las salidas y otra para el total), tenía que acceder a la sublista de las entradas, actualizar el elemento en el índice 0 de esa sublista (que guarda la cantidad de unidades de esa moneda en específico), para luego actualizar el índice 1 (que multiplica la cantidad de entradas * la denominación y así calcular la cantidad de dinero que eso representa) y luego ya estaría.

Esto fue muy complejo, pero llegado a este punto sería contraproducente devolverme y modificar todo. Por eso, pese a que ya conocía el tema de diccionarios, este predicamento fue de gran utilidad para repasar el manejo de diccionarios anidados; la complejidad que me impuse me ayudó a ganar experiencia con esta clase de estructuras de datos.

En conclusión, este proyecto fue un excelente ejercicio para introducirme al mundo de Tkinter y las interfaces gráficas. Si bien tuve un poco de fricción al comienzo, luego de haber experimentado todos los problemas y dificultades que este tercer trabajo implicó, me doy cuenta de lo familiarizado que estoy ahora de conceptos totalmente nuevos para mí hace un mes. Ahora sé cómo hacer ventanas, ponerles barras de menú, asociar funciones a *widgets* y botones, desplegar información en tiempo real a medida que otras variables van cambiando, validar datos de entrada... En fin, son innumerables las herramientas nuevas con las que ahora porto en mi camino hacia el futuro de esta carrera y los cursos siguientes.

4 Estadística de tiempos

Actividad Realizada	Horas
Análisis del problema	2h
Diseño de algoritmos	12 h
Investigación de: • GUI • Programación por eventos • Datetime	4h 50 mins 4h 45 mins 1h
Programación	62 h
Documentación interna	5h 30 mins
Pruebas	15 h 30 mins
Elaboración del manual de usuario	9 h
Elaboración de documentación del proyecto	2h 45min
Sesiones de tutorías para ayudar con problemas en el proceso	6h
TOTAL	131 horas y 30 minutos.

5 Lista de revisión del proyecto y análisis de resultados

Concepto	Puntos originales	Avance 100/%/0	Puntos obtenidos	Análisis de resultados
Barras de menú	5	100%		
Configuración	10	100%		
Dinero del cajero: cargar cajero	10	100%		
Dinero del cajero: saldo cajero	5	100%		
Entrada de vehículo	10	100%		
Cajero del estacionamiento: paso 1	10	100%		
Cajero del estacionamiento: paso 2	10	100%		
Cajero del estacionamiento: paso 3	5	100%		
Salida de vehículo	10	100%		
Reporte de ingresos: - Ingresos reales 6p - Ingresos estimados 6p	12	100%		
Ayuda (manual de usuario desplegado en el programa)	5	100%		
Leer datos de archivos	4	100%		
Grabar datos en archivos	4	100%		
TOTAL	100			
Partes desarrolladas adicionalmente: 1. En el reporte de ingresos de dinero, se despliegan los ingresos totales, pero en adición, los ingresos en efectivo y por tarjeta de crédito. 2. En el cajero, se agregó lógica para que se pueda anular el pago incluso pese a haber llegado al precio por pagar. 3. En la configuración, se facilita al usuario ver los datos registrados, autorellenando los campos si ya tienen valores. Igualmente se pueden modificar si la lista parqueo está vacía.		100%		

6 Manual del usuario

Nota: para evitar desajustes de formato con respecto al tamaño de página del documento del manual, así como números de página e hipervínculos para el índice, así como flechas y diagramas pegados encima de pantallazos adjuntados, en lugar de concatenar el manual con este presente documento, el manual se encuentra adjuntado por separado en la carpeta .zip de la entrega en el TecDigital.

7 Bibliografía

- Campos, O. (2011, julio 21). Introducción a la programación dirigida por eventos. *Desarrollo*. <https://www.genbeta.com/desarrollo/introduccion-a-la-programacion-dirigida-por-eventos>
- ES DESIGN. (2023, abril 21). ¿Qué es una interfaz gráfica de usuario o GUI? *Diseño web*. <https://www.esdesignbarcelona.com/actualidad/disenio-web/que-es-una-interfaz-grafica-de-usuario-gui>
- Ionos. (2020, setiembre 9). ¿Qué es una interfaz gráfica de usuario (GUI)? *Desarrollo web*. <https://www.ionos.com/es-us/digitalguide/paginas-web/desarrollo-web/que-es-una-gui/>
- Martín, G. (2020, junio 30). Interfaces gráficas en Python con Tkinter. *Tutoriales*. <https://adictosaltrabajo.com/2020/06/30/interfaces-graficas-en-python-con-tkinter/>
- Pillow. (s.f.). Pillow (PIL Fork) 10.3.0 documentation. <https://pillow.readthedocs.io/en/stable/>
- Programación Desde Cero. (s.f.). Que es un evento en programación orientada a objetos. *Programación Orientada a Objetos*. <https://programacion.top/orientada-objetos/evento/>
- Python Software Foundation. (s.f.). datetime — Tipos básicos de fecha y hora. *Tipos de datos*. <https://docs.python.org/es/3.9/library/datetime.html>
- Python Software Foundation. (s.f.). Interfaces gráficas de usuario con Tk. *La biblioteca estándar de Python*. <https://docs.python.org/es/3/library/tk.html>
- Python Software Foundation. (s.f.). tkinter — Python interface to Tcl/Tk. *Interfaces gráficas de usuario con Tk*. <https://docs.python.org/es/3/library/tkinter.html>